GPT2-Clone

Public

1 Branch 0 Tags Go to file t Go to file Add file Code

iqbal-sk Update README.md to provide comprehensive project overview and usa...

2726190 · last month

modules	Corrected wrong implementati...	8 months ago
utils	Implemented utility function fo...	8 months ago
GPTModel.py	Implemented GPT Class	8 months ago
README.md	Update README.md to provid...	last month
requirements.txt	Added requirements.txt	8 months ago
train_gutenberg.py	Included	8 months ago

About

Hands-on GPT-2 built from scratch—clean Transformer blocks, practical training pipeline, and inference tools—showcasing ML systems craftsmanship.

#deep-learning #cuda #transformers #text-generation #pytorch #language-model #attention-mechanism #gpt2

Readme

Activity

0 stars

README

GPT-2 Clone (From Scratch, PyTorch)

A clean, from-scratch implementation of a GPT-2–style Transformer language model in PyTorch, including all core building blocks (Multi-Head Attention, GELU MLP, LayerNorm, causal masking), a data pipeline, configurable training loop with warmup+cosine decay, checkpointing, and text generation utilities. Trains on plain `.txt` corpora (e.g., Project Gutenberg) and logs metrics/samples to Weights & Biases.

This project showcases systems-level deep learning skills: implementing core transformer components correctly and readably, designing a practical training loop, and exposing ergonomic utilities for evaluation and inference.

Highlights

- Minimal, readable implementation of GPT-2 components without heavy frameworks.
- Configurable model sizes, including a debug-friendly small config and a 355M-style setup.
- Tokenization via `tiktoken` (GPT-2 BPE), end-of-text handling, and sliding-window dataset creation.
- Training loop with AdamW, decoupled weight decay (parameter-wise groups), linear warmup, cosine annealing, and gradient clipping.
- Checkpointing (interrupt-safe), validation, and sample generation during training.
- Device support for CUDA, Apple Silicon (MPS), and CPU.
- W&B integration out-of-the-box for metrics, gradients, and sample logging.

What This Project Does

1. Implements a GPT-2-like model end-to-end:

- Token and positional embeddings, N× Transformer blocks, final LayerNorm, tied output head.
- Multi-Head Attention uses PyTorch's `scaled_dot_product_attention` with causal masking.
- Feed-forward MLP with GELU, residual connections, and dropout.

2. Provides a practical training/evaluation pipeline:

- Text dataset windows created via a stride over token IDs for efficient batching.

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Languages

Python 100.0%

Suggested workflows

Based on your tech stack

Pylint Configure

Lint a Python application with pylint.

SLSA Generic generator Configure

Generate SLSA3 provenance for your existing release workflows

Python package Configure

Create and test a Python package on multiple Python

- Per-book train/val split, periodic evaluation and plotting of losses.
- Learning rate schedule: linear warmup then cosine decay; gradient clipping after warmup.
- Checkpoints saved periodically and at the end; supports resume.

3. Includes simple, flexible text generation utilities:

- Greedy decoding, temperature scaling, top-k sampling, and early stop on EOS.

Repository Structure

```
GPTModel.py                # GPT-2 style model (embeddings, blocks, head)
modules/
  layers/
    multi_head_attention.py # MHA with causal SDPA
    feed_forward.py         # GELU MLP
    layer_norm.py           # Custom LayerNorm
    gelu.py                 # GELU activation
  data/
    Datasets.py             # GPTDataset + DataLoader factory (tiktoken)
utils/
  training.py               # Train/val dataloader helpers
  loss.py                   # Loss, evaluation, and plotting
  generation.py             # Encoding/decoding and sampling utilities
train_gutenberg.py          # End-to-end training script w/ W&B
requirements.txt
```

Quick Start

1) Setup

```
python -m venv .venv && source .venv/bin/activate  # optional
pip install -r requirements.txt
# Optional but recommended for logging
wandb login
```

2) Prepare Data

- Place plain text files under a directory (default: `data/sample_train_data`).
- For Project Gutenberg or similar corpora, ensure license compliance and preprocess into `.txt` files.

Example layout:

```
data/
  sample_train_data/
    book1.txt
    book2.txt
    ...
```

3) Train

Debug-sized model (fast iteration):

```
python train_gutenberg.py \
  --data_dir data/sample_train_data \
  --output_dir model_checkpoints \
  --n_epochs 1 \
  --batch_size 4 \
  --lr 1e-4 \
  --debug True
```

Larger 355M-style model:

```
python train_gutenberg.py \
  --data_dir data/sample_train_data \
  --output_dir model_checkpoints \
  --n_epochs 1 \
  --batch_size 4 \
  --lr 1e-4
```

Key flags:

- `--data_dir`: Directory with `.txt` files.
- `--output_dir`: Where to save checkpoints and plots.
- `--n_epochs`, `--batch_size`, `--lr`: Usual training knobs.
- `--debug`: Switches to a smaller, quicker model config.
- `--print_sample_iter`: Iterations between sample generations during training.
- `--eval_freq`: Evaluation cadence (train/val loss).
- `--save_ckpt_freq`: Checkpoint save cadence.
- `--load_model_path` / `--load_optimizer_path`: Resume from checkpoints.

4) Resume Training

```
python train_gutenberg.py \
  --data_dir data/sample_train_data \
  --output_dir model_checkpoints \
  --load_model_path model_checkpoints/checkpoint_000100.pt \
  --load_optimizer_path model_checkpoints/checkpoint_000100.pt
```

5) Generate Text (Standalone)

```
import torch, tiktoken
from GPTModel import GPTModel
from utils.generation import text_to_token_ids, token_ids_to_text, generate

# Match the config used for training
GPT_CONFIG_355M = {
    "vocab_size": 50257,
    "context_length": 1024,
    "emb_dim": 1024,
    "n_heads": 16,
    "n_layers": 24,
    "drop_rate": 0.1,
    "qkv_bias": False,
}

device = torch.device("cuda" if torch.cuda.is_available() else ("mps" if torch.backends.mps.is_available() else "cpu
model = GPTModel(GPT_CONFIG_355M).to(device)

# Load a trained checkpoint
ckpt = torch.load("model_checkpoints/final_checkpoint.pt", map_location=device)
model.load_state_dict(ckpt["model_state_dict"])
model.eval()

tokenizer = tiktoken.get_encoding("gpt2")
prompt = "Every effort moves you"
idx = text_to_token_ids(prompt, tokenizer).to(device)

with torch.no_grad():
    out = generate(model, idx, max_new_tokens=80, context_size=GPT_CONFIG_355M["context_length"], temperature=0.8, t
print(token_ids_to_text(out, tokenizer))
```

Model And Training Details

- Architecture: Token/position embeddings → N× {LayerNorm → MHA (causal) → Dropout → Residual, LayerNorm → MLP(GELU) → Dropout → Residual} → final LayerNorm → linear vocab head.

- Attention: Uses `torch.nn.functional.scaled_dot_product_attention(..., is_causal=True)` for correctness and efficiency.
- Optimizer: AdamW with decoupled weight decay; parameters are grouped so weight decay applies only to matrix-like tensors (`ndim >= 2`).
- Schedule: Linear warmup → cosine decay; gradient clipping after warmup.
- Tokenization: `tiktoken` GPT-2 BPE with `<|endoftext|>` support.
- Evaluation: Periodic train/val cross-entropy; loss curves saved to `losses.pdf` in `output_dir`.
- Devices: CUDA, Apple MPS, or CPU are auto-detected.

Installation Notes

- Python 3.10+ recommended.
- PyTorch 2.0+ required (for SDPA and performance). Install the wheel appropriate for your platform.
- `wandb` is optional but recommended; run `wandb login` before training to enable logging.

Why This Project Stands Out

- From-scratch correctness: Each building block is implemented explicitly for clarity, with readable tensor shapes and minimal abstraction.
- Practical training pipeline: Warmup+cosine schedule, gradient clipping, and checkpointing mirror modern LLM training best practices.
- Ergonomic utilities: Tokenization helpers, sampling strategies, and progress ETA make the code pleasant to use and extend.
- Clean, modular code: Easy to adapt to different model sizes or datasets.

Roadmap / Ideas

- Add byte-level tokenization fallback and Unicode normalization utilities.
- Add mixed precision (AMP) + gradient accumulation for larger effective batch sizes.
- Weight tying between token embeddings and output head to reduce params.
- Better dataset builders and streaming loaders for massive corpora.
- Unit tests and micro-benchmarks for attention kernels.

Acknowledgements

- Inspired by the GPT-2 architecture (Radford et al.) and modern training practices.
- Uses `tiktoken` for GPT-2 BPE and W&B for experiment tracking.